```
const INPUT_IDS = [
  'weight', 'diameter', 'dragCoeff', 'liftCoeff',
  'angle', 'velocity', 'yawAngle', 'launchHeight',
  'windX', 'windY', 'targetHeight', 'airDensity'
];

function saveSettings() {
  INPUT_IDS.forEach(id => {
    const el = document.getElementById(id);
    if (el) localStorage.setItem('arrow_sim_' + id, el.value);
  });
}

function loadSettings() {
  INPUT_IDS.forEach(id => {
    const savedValue = localStorage.getItem('arrow_sim_' + id);
    const el = document.getElementById(id);
    if (el && savedValue !== null) {
      el.value = savedValue;
    }
  });
}

function switchPanel(type) {
  saveSettings();
  const panels = ['arrow', 'method', 'env', 'result'];
  panels.forEach(p => {
    const el = document.getElementById('panel-' + p);
    if (el) el.classList.remove('active');
  });

  const targetPanel = document.getElementById('panel-' + type);
  if (targetPanel) {
    targetPanel.classList.add('active');
  }
  updateTabActiveStyle(type);
  if (typeof drawScene === 'function') drawScene();
}

function updateTabActiveStyle(type) {
  const tabItems = document.querySelectorAll('.tab-bar .tab-item');
  tabItems.forEach(item => item.classList.remove('active'));
  const typeOrder = ['arrow', 'method', 'env', 'result'];
```

```
  const activeIndex = typeOrder.indexOf(type);
  if (activeIndex !== -1 && tabItems[activeIndex]) {
    tabItems[activeIndex].classList.add('active');
  }
}

let currentView = 'side';
function changeView(viewType, element) {
  const buttons = document.querySelectorAll('.segmented-control .segment-btn');
  buttons.forEach(btn => btn.classList.remove('active'));
  if (element) element.classList.add('active');
  currentView = viewType;
  if (typeof drawScene === 'function') drawScene();
}

const NEGATIVE_ALLOWED_IDS = ['angle', 'yawAngle', 'windX', 'windY', 'targetHeight'];

window.addEventListener('DOMContentLoaded', () => {
  loadSettings();
  INPUT_IDS.forEach(id => {
    const el = document.getElementById(id);
    if (el) {
      el.addEventListener('input', () => {
        if (NEGATIVE_ALLOWED_IDS.includes(id)) {
          let val = el.value;
          val = val.replace(/[^0-9.-]/g, '');
          val = val.replace(/(?!^)-/g, '');
          const parts = val.split('.');
          if (parts.length > 2) {
            val = parts[0] + '.' + parts.slice(1).join('');
          }
          el.value = val;
        }
        saveSettings();
        if (typeof drawScene === 'function') drawScene();
      });
    }
  });

  //
==========================================================
================
  // [💡 위치 기억 기능 추가] 무제한 스크린 프리 드래그 제어 시스템
```

```
  //
==========================================================
================
  const dragBtn = document.getElementById('draggableFireBtn');
  if (!dragBtn) return;

  let isDragging = false;
  let hasMoved = false;
  let startX = 0, startY = 0;
  let initialLeft = 0, initialTop = 0;

  // 이전 사용 위치 복원 함수
  function loadButtonPosition() {
    const savedLeft = localStorage.getItem('arrow_sim_btn_left');
    const savedTop = localStorage.getItem('arrow_sim_btn_top');

    if (savedLeft !== null && savedTop !== null) {
      dragBtn.style.left = savedLeft;
      dragBtn.style.top = savedTop;
      dragBtn.style.right = 'auto'; // 초기 CSS 우측 고정 해제
    }
  }

  function startDrag(e) {
    isDragging = true;
    hasMoved = false;
    const clientX = e.touches ? e.touches[0].clientX : e.clientX;
    const clientY = e.touches ? e.touches[0].clientY : e.clientY;
    startX = clientX;
    startY = clientY;
    const rect = dragBtn.getBoundingClientRect();
    initialLeft = rect.left;
    initialTop = rect.top;
  }

  function doDrag(e) {
    if (!isDragging) return;
    const clientX = e.touches ? e.touches[0].clientX : e.clientX;
    const clientY = e.touches ? e.touches[0].clientY : e.clientY;
    const deltaX = clientX - startX;
    const deltaY = clientY - startY;
    if (Math.abs(deltaX) > 4 || Math.abs(deltaY) > 4) {
      hasMoved = true;
```

```
    }
    let newLeft = initialLeft + deltaX;
    let newTop = initialTop + deltaY;

    // 브라우저 뷰포트 전체 크기를 기준으로 가둠
    const maxLeft = window.innerWidth - dragBtn.offsetWidth;
    const maxTop = window.innerHeight - dragBtn.offsetHeight;
    newLeft = Math.max(0, Math.min(newLeft, maxLeft));
    newTop = Math.max(0, Math.min(newTop, maxTop));

    dragBtn.style.left = newLeft + 'px';
    dragBtn.style.top = newTop + 'px';
    dragBtn.style.right = 'auto';
  }

  function endDrag(e) {
    if (!isDragging) return;
    isDragging = false;
    if (!hasMoved) {
      if (typeof fireArrow === 'function') fireArrow();
    } else {
      // 드래그 이동이 정상적으로 완료되면 로컬스토리지에 위치 좌표 저장
      localStorage.setItem('arrow_sim_btn_left', dragBtn.style.left);
      localStorage.setItem('arrow_sim_btn_top', dragBtn.style.top);
    }
  }

  // 초기 실행 시 저장된 버튼 위치 불러오기
  loadButtonPosition();

  // 이벤트 리스너 등록
  dragBtn.addEventListener('touchstart', startDrag, { passive: true });
  window.addEventListener('touchmove', doDrag, { passive: false });
  window.addEventListener('touchend', endDrag);
  dragBtn.addEventListener('mousedown', startDrag);
  window.addEventListener('mousemove', doDrag);
  window.addEventListener('mouseup', endDrag);
});
// 인트로 공지사항 모달 닫기 함수
function closeIntro() {
  const introModal = document.getElementById('introModal');
  if (introModal) {
    introModal.style.opacity = '0';
```

```
    introModal.style.visibility = 'hidden';
    setTimeout(() => {
      introModal.style.display = 'none';
    }, 300); // CSS transition 시간(0.3s)과 일치시켜 부드럽게 제거
  }
}
```